

Phase 2 and Phase 3 — Explained in Simple Words

IntelliBot / IntelliWarehouse - Robotic Intelligence project - checklist Phases 2 and 3 (Person A)

This file explains **what was built in Phase 2 and Phase 3**, **why**, **what results we got**, and **how to run everything yourself**. It is written for someone who is new to the project and new to Webots. Read it top to bottom once before the evaluation.

1. The project in one paragraph

We are building **IntelliBot**, a small robot that works in a pretend warehouse. It gets a job like *"pick the blue package P3 and bring it to Zone B"*. To do the job, it has to **look** for the package with its camera, **plan** a route with the A* algorithm, **drive** along that route, **avoid** bumping into things with its LiDAR, and finally **deliver** the package. Everything happens inside a computer simulation; there is no real robot.

2. What is Webots? (basic idea)

Webots is a free program that simulates robots. Think of it as a video game with real physics: gravity, friction, wheels that roll and can slip.

Important words:

Word	Simple meaning
World (<code>.wbt</code> file)	A text file that describes everything in the scene: floor, walls, boxes, lights, and the robot. Our world is <code>worlds/warehouse.wbt</code> .
Node	One "thing" inside the world file, e.g. <code>Solid</code> (a physical object), <code>Shape</code> (what it looks like), <code>Robot</code> , <code>Camera</code> , <code>Lidar</code> .
boundingObject	The invisible shape used for collisions . If an object has no boundingObject, other things pass straight through it.
physics	Gives an object mass so it can fall, be pushed, roll. Objects without physics never move (good for walls and shelves).
Controller	A normal Python program that "is the brain" of the robot. It reads sensors and sets motor speeds. It lives in <code>controllers/<name>/<name>.py</code> . The robot's <code>controller</code> field in the world says which one to run.
Time step	Webots moves the simulation forward in small steps. Ours is 16 ms (about 60 steps per second). Every step the controller reads sensors and sends motor commands.
Supervisor	A special robot permission that lets the controller see and move <i>anything</i> in the world (e.g. read the true position of a box). We use it for checking and screenshots, not for cheating in the mission.
Simulation time	The clock shown in the Webots toolbar. In "fast" mode it runs much faster than real time.

Devices on our robot

Device	Name in code	What it gives
2 wheel motors	<code>left_motor</code> , <code>right_motor</code>	We set wheel speed (rad/s).
2 wheel encoders	<code>left_encoder</code> , <code>right_encoder</code>	How much each wheel has turned (for odometry).
LiDAR	<code>lidar</code>	360 distance readings around the robot (0.05–4 m).
Camera	<code>camera</code>	320×240 color picture from the front.
GPS	<code>gps</code>	The true (x, y) position — perfect in simulation.
InertialUnit (IMU)	<code>imu</code>	The robot's angle (yaw θ).

Coordinate system (ENU): x points east (right on the top view), y points north (up on the top view), z points up. Angle $\theta = 0$ means the robot faces +x; turning left makes θ bigger.

3. Who does what (team split)

Person	Area	Phases 2–3 belong to
A — Simulation & Robot	world, robot, kinematics, PID control	[OK] Phase 2 and Phase 3 are Person A's work
B — Perception	camera, LiDAR, OpenCV detection	Phase 4
C — Planning & Decisions	grid, A*, state machine, integration	Phases 1, 6

4. Phase 2 — The warehouse world

2.1 World generator (`tools/generate_world.py`)

What: Instead of building the world by hand in Webots, a Python script **writes the world file automatically** from the numbers in `config.py` (positions and sizes of shelves, crates, packages, zones, robot).

Why: The A* planner also reads `config.py` to build its map. So the map the planner uses and the world the robot drives in **can never disagree**. Change a shelf in `config.py`, re-run the script, and both update.

What is in the world (8 m × 6 m):

- gray floor and 4 walls
- 4 dark shelves (1.6 m long)
- 2 brown crates (static obstacles)
- a light-gray pickup area with 3 packages: **P1 red, P2 green, P3 blue** (pure colors so the camera can find them easily)
- **Zone A yellow** and **Zone B cyan** (flat floor patches, no collision)
- one extra crate parked in the top-right corner (`DYN_OBSTACLE`) — it will be moved into the robot's path in the final evaluation to show re-planning
- the robot in the middle at (0, -0.3), facing east
- **two lights with no shadows** (shadows would confuse the color detector)

It writes **two world files** that are the same except for the robot's brain:

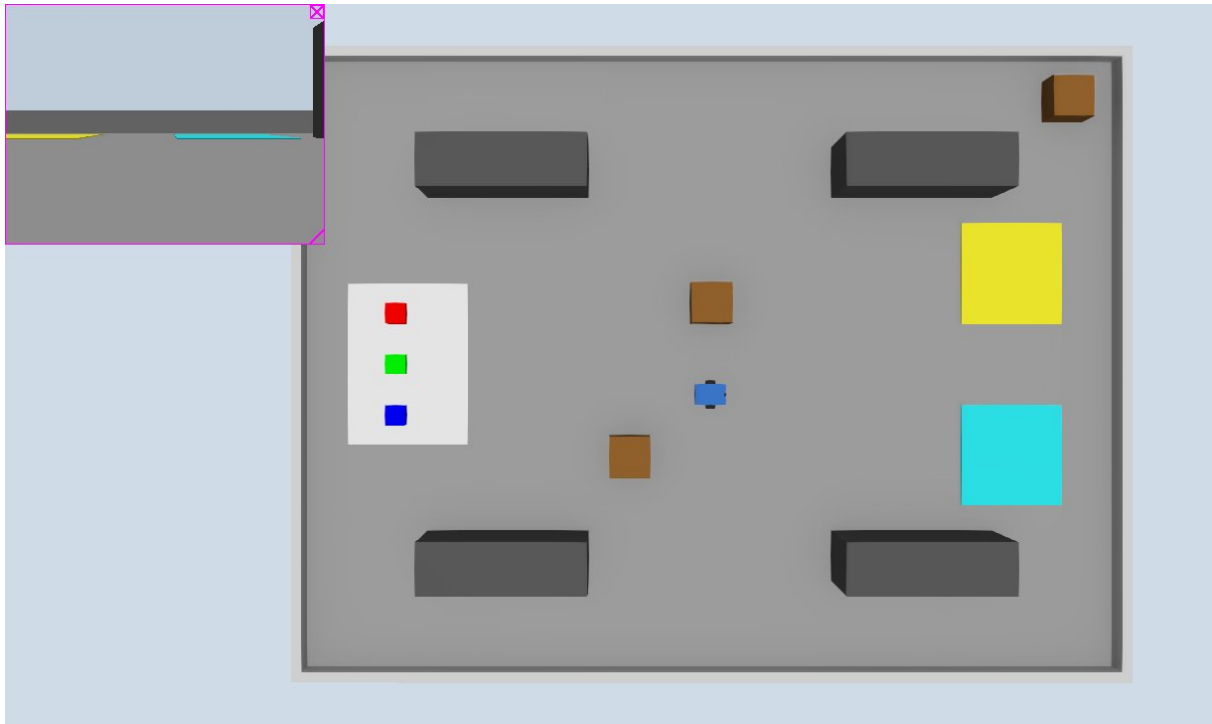
- `worlds/warehouse.wbt` → runs `intellibot_controller` (the full mission)
- `worlds/warehouse_drive_test.wbt` → runs `drive_test` (the tests for Phase 2–3)

2.2 Screenshots

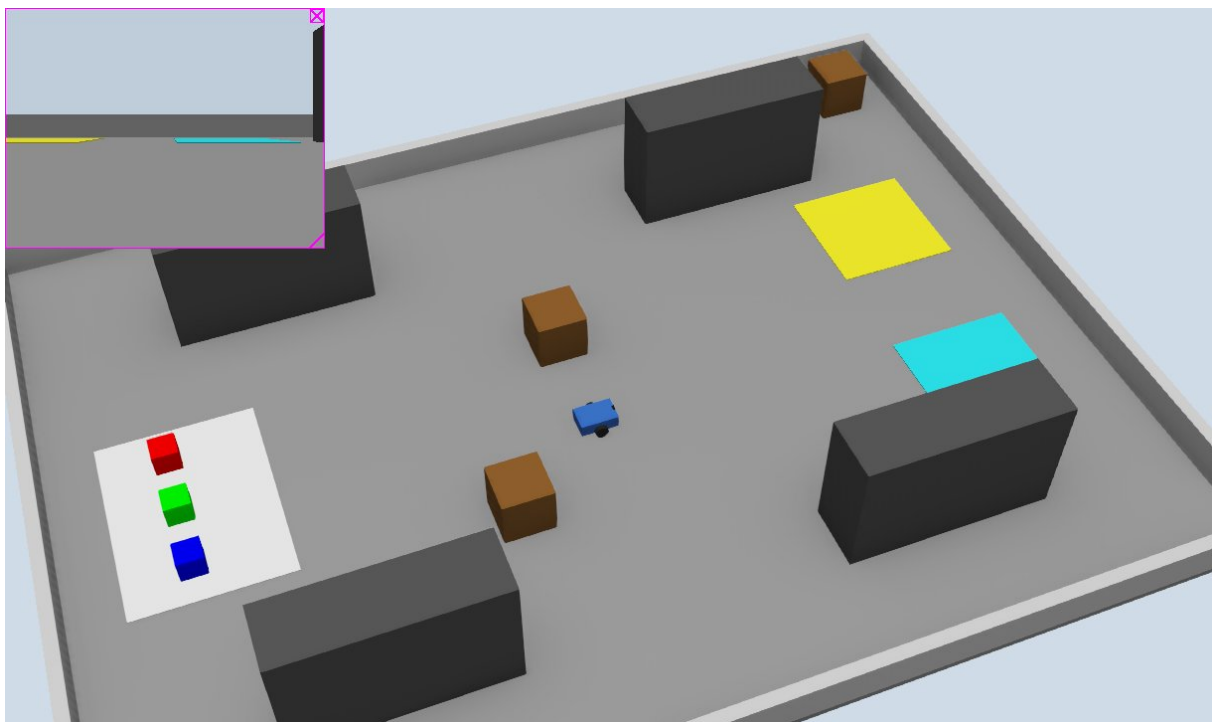
The `drive_test` controller has a `snapshot` mode. It moves the Webots camera (the "Viewpoint") using the Supervisor and saves pictures:

- `docs/evidence/world_topdown.png` — view from straight above
- `docs/evidence/world_angle.png` — 3D view from the corner

(The small picture in the top-left corner of the screenshots is the **robot's own camera view**.)



Top-down view of the generated warehouse (robot camera view top-left).



3D view: shelves (dark), crates (brown), packages P1-P3, Zone A (yellow), Zone B (cyan).

2.3 World vs config check

The **worldcheck** mode waits 1 second (so things settle under gravity), then uses the Supervisor to read the **real** position of every package, shelf and crate and compares it with **config.py**. If anything is more than 1 cm off it prints a warning.

Result: [WORLD CHECK] world matches config [OK]

5. Phase 3 — Robot, driving and kinematics

3.1 The robot

Built from basic Webots nodes (no ready-made robot), so we fully understand it:

- a blue box body: 30 cm × 20 cm × 10 cm, 2 kg
- **2 driven wheels** (radius $r = 5$ cm), $L = 24$ cm apart, each with a motor and an encoder — this is called a **differential-drive** robot (like a wheelchair: turning happens by spinning the two wheels at different speeds)
- **2 small ball casters** (front and back) with **zero friction** so they just slide and hold the robot level
- LiDAR on top, camera on the front, GPS and IMU inside

Stability test (stability mode): motors at 0 for 10 seconds.

Result: moved **0.00 mm**, tilted **0.000°** → the robot rests perfectly. **[OK]**

3.2 Drive test (sequence mode)

The robot runs a fixed script: forward 2 s → spin left 2 s → arc 3 s → reverse 1.5 s → stop. It prints its position every second.

Result: going forward increases x ; spinning left increases θ . This **confirms our coordinate conventions** are right. **[OK]**

3.3 Kinematics validation

Kinematics = the math that connects wheel speeds to robot motion.

Forward kinematics (wheels → robot):

$$\begin{aligned} v &= r/2 \times (\omega_R + \omega_L) && \text{(forward speed, m/s)} \\ \omega &= r/L \times (\omega_R - \omega_L) && \text{(turning speed, rad/s)} \end{aligned}$$

Inverse kinematics (robot → wheels) — this is what the controller uses:

$$\begin{aligned} \omega_R &= (v + \omega \cdot L/2) / r \\ \omega_L &= (v - \omega \cdot L/2) / r \end{aligned}$$

Example: we want $v = 0.2$ m/s and $\omega = 0.5$ rad/s → $\omega_L = (0.2 - 0.5 \times 0.12) / 0.05 = \mathbf{2.8 \text{ rad/s}}$, $\omega_R = (0.2 + 0.06) / 0.05 = \mathbf{5.2 \text{ rad/s}}$.

Test (kinematics mode): For 4 commands we compute wheel speeds with the inverse formula, drive for 4 seconds, and **measure** the real v and ω from GPS and IMU. If the math is right, measured $\approx$ commanded.

Final result (docs/evidence/kinematics_validation.md):

Command (v, ω)	Wheels (ω_L, ω_R)	Measured (v, ω)	Error
0.2 m/s, 0	4.0, 4.0	0.198, 0	-0.9 %
0, 1.0 rad/s (spin)	-2.4, +2.4	0, 0.991	-0.9 %
0.2, 0.5 (curve)	2.8, 5.2	0.198, 0.495	-0.9 %
-0.1, 0 (reverse)	-2.0, -2.0	-0.099, 0	0.9 % slower

All errors are **below 1 %**. **[OK]** (The small -0.9 % is because the wheel sinks about 0.4 mm into the "soft" simulated floor, making its effective radius a tiny bit smaller.)

3.4 Odometry vs ground truth

Odometry = the robot guessing its own position **only from how much its wheels turned** (encoders). Real robots do this because they don't have perfect GPS. Small errors add up over time ("drift").

$$\begin{aligned} \text{Each step: } ds_L &= r \cdot \Delta\phi_L, \quad ds_R = r \cdot \Delta\phi_R && \text{(distance each wheel rolled)} \\ ds &= (ds_R + ds_L)/2, \quad \Delta\theta = (ds_R - ds_L)/L \end{aligned}$$

```

x += ds*cos(theta + delta_theta/2)
y += ds*sin(theta + delta_theta/2)
theta += delta_theta

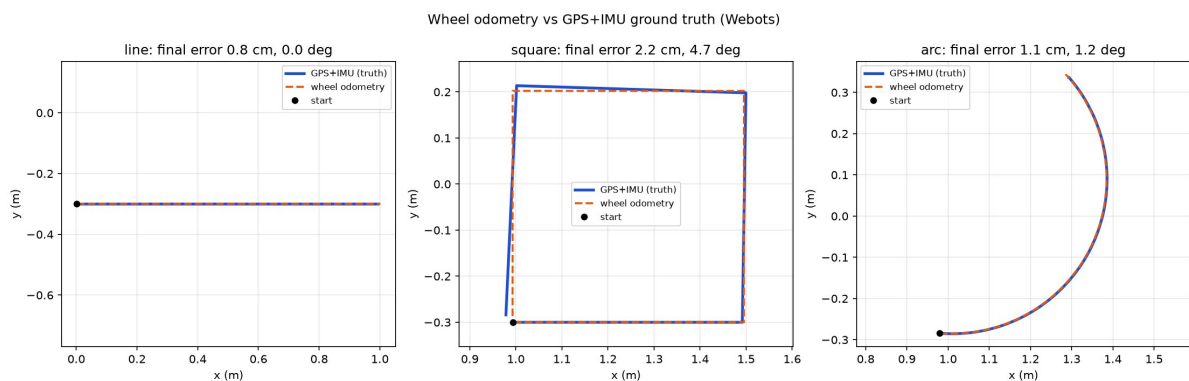
```

Test (odometry mode): drive a straight line, a square, and an arc; compare odometry with GPS+IMU (the truth).

Result ([docs/evidence/odometry_vs_gps.png](#)):

Path	Distance	Final error
Line	1 m	0.8 cm
Square	2 m	2.2 cm and 4.7°
Arc	0.9 m	1.1 cm and 1.2°

The square has the most drift because it has 4 turns, and turning is where wheels slip the most. [OK]



Wheel odometry (orange dashed) vs GPS+IMU truth (blue) for line, square and arc.

3.5 Pen trail — not done

This was optional (P2). It would draw the robot's path on the floor. It needs a textured floor, so we skipped it.

6. Problems we found and fixed (great for "what challenges did you face?")

These came up the first time we ran the world in Webots. Each one is a good answer to an evaluation question.

- Things would fall through the floor.** The floor had no `boundingObject`, so it had no collision shape. **Fix:** gave the floor a collision box.
- The world file would not load.** The first line must be `#VRML_SIM R2025a utf8` — the word `utf8` was missing. **Fix:** added it.
- Wrong place for friction settings.** The "zero friction for casters" setting was inside the robot, but Webots only accepts it in `WorldInfo`. **Fix:** moved it, and gave the casters a material called `"caster"`.
- Casters did nothing.** A small Solid without `physics` is ignored for collisions (Webots printed a warning). **Fix:** gave each caster a tiny mass.
- [*] The robot turned 14 % too slowly (the most interesting one).** Forward speed was perfect, but every turn was only 86 % of what we asked.
 - Thinking:** $0.24 \text{ m} \div 0.858 = 0.28 \text{ m}$. That is exactly $L + \text{wheel width}$ ($0.24 + 0.04$). So the robot was acting as if the wheels were 28 cm apart, not 24 cm.
 - Reason:** a cylinder-shaped wheel touches the floor at its **outer edge**, not its middle.
 - Proof:** we made the wheels half as wide $\rightarrow$ the error became half (-7.6%).

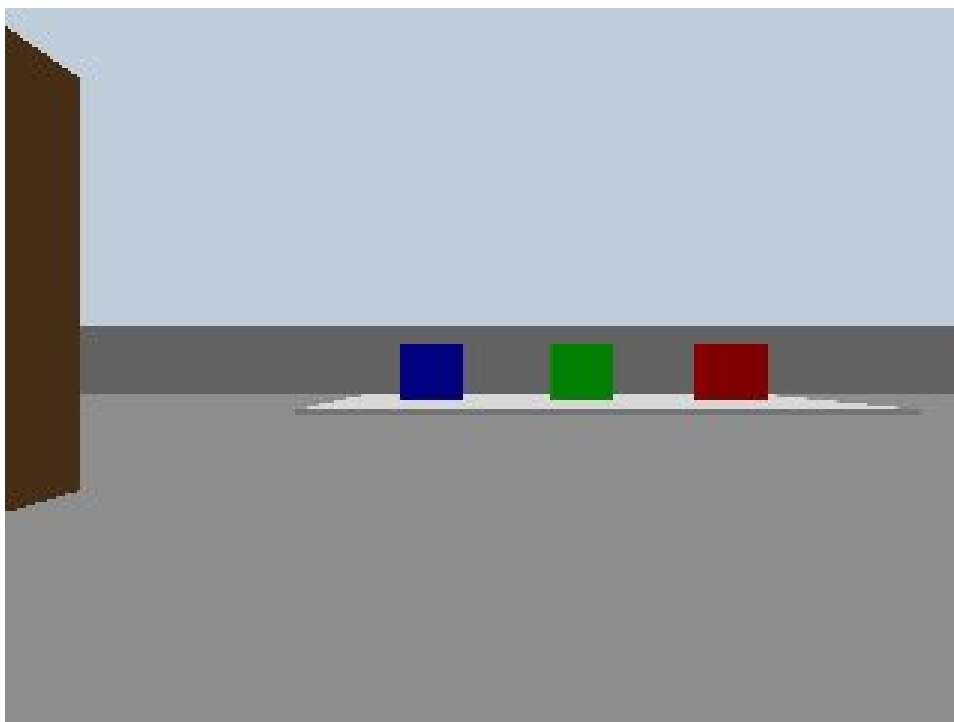
- **Fix:** the wheel's collision shape is now a **sphere** with the same radius (it still *looks* like a cylinder). A sphere touches the floor at exactly one point in the middle → error dropped to **-0.9 %**, and square odometry drift dropped from **35 cm to 2.2 cm**.

6 **The spare crate was inside the wall** → moved it to a free corner.

7 **The mission controller could not start.** We had a file called `controller.py`, but Webots' own Python library is also called `controller`. Python loaded our file instead of Webots'. **Fix:** renamed ours to `waypoint_follower.py`.

8 **The robot could not see the packages.** With one light, the sides of the packages facing the robot were in the dark (almost black), so the color detector ignored them. We found this with a new `capture` mode that saves what the camera sees (`docs/evidence/frames/`). **Fix:** added a second light from the opposite side.

Also fixed in the test code: the CSV files had broken headers (`writerows` instead of `writerow`), and the old measurement method gave the wrong sign when reversing and broke when the robot spun more than 180°.



What the robot camera sees when facing the pickup area (after the lighting fix): P3 blue, P2 green, P1 red.

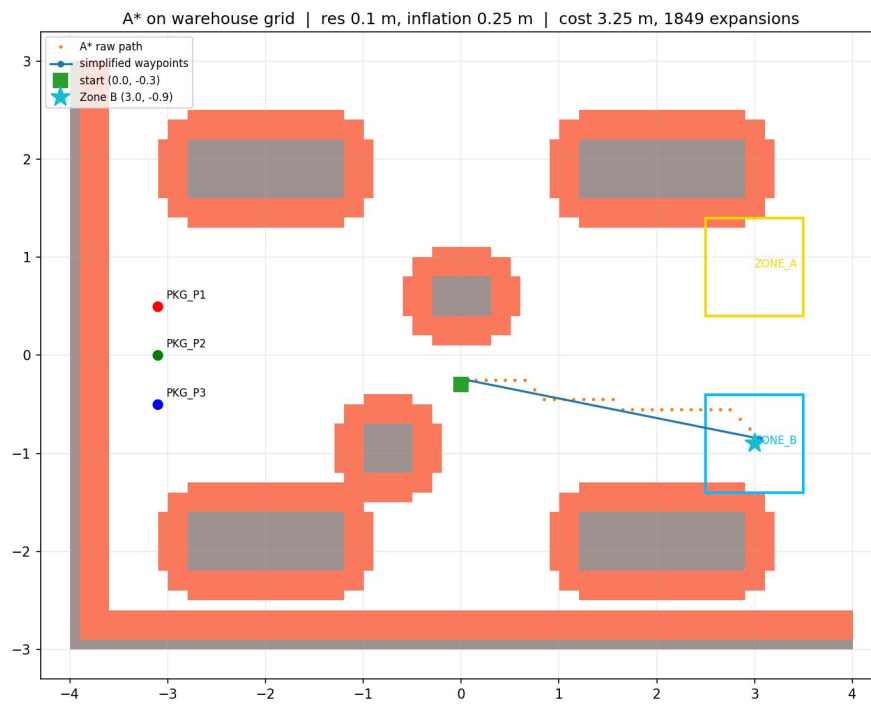
7. Bonus: first full mission run

After the fixes we ran the complete mission once (this is really Phase 6, done early to check that everything connects):

```
t=0.0 s   Task: pick P3 (blue), deliver to Zone B
t=4.7 s   Package P3 detected (camera + LiDAR)      -> A* path 10 waypoints, 2.10 m
t=20.5 s  Reached the package                       -> PICK (placeholder)
t=21.5 s  Plan delivery                             -> A* path 19 waypoints, 5.04 m
t=61.7 s  Reached Zone B -> DONE
MISSION REPORT: time 61.7 s, path length 7.14 m, replans 0
```

Be honest about these two things if asked:

- The package position was estimated **0.6 m too close** (the robot thought P3 was at $x = -2.50$, really it is at -3.10). We think the LiDAR's ray order (left-to-right vs right-to-left) is reversed in our code. Checking this is Phase 4.2 (Person B).
- **Pickup is only a pause** for now. Really attaching the package is a final-evaluation feature.



A* plan on the inflated occupancy grid (from Phase 1).

8. How to run everything yourself (step by step)

Step 0 — One-time setup

- 1 Install **Webots R2025a** (on this PC it is in **D:\RI_CyberBotics\Webots**).
- 2 Install **Python 3.9 or newer**, then in a terminal in the project folder:

```
pip install -r requirements.txt
```


(installs numpy, opencv-python, matplotlib, pytest)
- 3 In Webots: **Tools** → **Preferences** → **General** → **Python command**. It must be the Python that has these packages. Here it is simply **python**. If you see **ModuleNotFoundError: cv2** in the Webots console, this setting is wrong — put the full path to **python.exe**.

Step 1 — (Only if you changed **config.py**) rebuild the world

```
python tools/generate_world.py
```

Step 2 — Look at the world and watch the robot (easiest way)

- 1 Open Webots → **File** → **Open World...**
- 2 Choose **E:\RI_Project\RI-Project\worlds\warehouse_drive_test.wbt**
- 3 Press ► (**Play**). The robot drives the ~10-second test sequence and prints its position in the console at the bottom.
- 4 Press **Reset** (the |◀ button) then ► to watch again.

To watch the **full mission** instead, open **worlds\warehouse.wbt** and press ► (takes about 1 minute of simulation time; it closes Webots at the end).

Mouse in the 3D view: left-drag = rotate, right-drag = move, scroll = zoom. If Webots asks "save changes?" when closing, click **No** (the world file is generated — don't edit it by hand).

Step 3 — Run a specific test mode (command line)

Open **Git Bash** in **E:\RI_Project\RI-Project**:

```
W="/d/RI_CyberBotics/Webots/msys64/mingw64/bin/webots.exe"

# watch a test in the Webots window (Webots closes by itself when done)
DRIVE_TEST_MODE=kinematics "$W" --mode=realtime worlds/warehouse_drive_test.wbt

# or run it fast in the background and only read the console text
DRIVE_TEST_MODE=kinematics "$W" --batch --mode=fast --stdout --stderr --minimize worlds/warehouse_drive_test.wbt
```

Mode	Checklist	What it does / writes
worldcheck	2.3	prints world matches config
snapshot	2.2	saves the two screenshots (needs the window, so no --minimize)
stability	3.1	10 s standing still → logs/stability.csv
sequence	3.2	forward / spin / arc / reverse, prints pose
kinematics	3.3	→ logs/kinematics_test.csv
odometry	3.4	→ logs/odometry.csv
capture	(debug)	spins and saves camera frames → docs/evidence/frames/

PowerShell version:

```
$env:DRIVE_TEST_MODE = "odometry"
& "D:\RI_CyberBotics\Webots\msys64\mingw64\bin\webots.exe" --mode=realtime worlds\warehouse_drive_test.wbt
Remove-Item Env:DRIVE_TEST_MODE
```

(If you forget the last line, opening Webots normally will also run that mode.)

Step 4 — Make the tables and the plot

After running `kinematics` and/or `odometry`:

```
python tools/analyze_logs.py
```

This updates `docs/evidence/metrics.md`, `kinematics_validation.md` and `odometry_vs_gps.png`.

Step 5 — Unit tests (no Webots needed)

```
python -m pytest tests/ -q          # expected: 63 passed
```

9. Likely evaluation questions — with simple answers

Q: Why did you make the world with a Python script instead of the Webots editor? So the world and the planner's map come from the same numbers (`config.py`) and can never be different. One change updates both.

Q: What is a differential-drive robot? A robot with two wheels on one axle, each with its own motor. Same speed → straight. Different speeds → it turns. Opposite speeds → it spins on the spot.

Q: Explain your kinematics equations. Forward speed is the average of the two wheel speeds times the radius: $v = r(\omega_R + \omega_L)/2$. Turning speed is the difference divided by the wheel distance: $\omega = r(\omega_R - \omega_L)/L$. We use the inverse to turn a desired (v, ω) into wheel speeds.

Q: How did you check that your kinematics is correct? We commanded 4 different (v, ω) values, measured the real motion with GPS and IMU, and compared. All errors are under 1 %.

Q: What is odometry and why does it drift? Estimating position from wheel rotation only. Every small slip or rounding error adds up and is never corrected, so the error grows over time. We measured 0.8–2.2 cm after 1–2 m.

Q: If you have GPS, why do you need odometry? In simulation GPS is perfect, but real robots indoors usually have no GPS. We use GPS as the "truth" to measure how good odometry is.

Q: What was the hardest problem? The robot turned 14 % too slowly. We noticed $0.24/0.858 = 0.28 =$ wheel distance + wheel width, so the wheels were touching the floor at their outer edge. Halving the wheel width halved the error, which proved it. We changed the wheel's collision shape to a sphere and the error fell below 1 %.

Q: Why do the casters have zero friction? They only support the robot. If they had friction, they would fight the wheels when turning and make the motion wrong.

Q: Why no shadows and why two lights? Our package detector works by color. Shadows and dark faces change the color values, so the detector misses packages. Two opposite lights light every side.

Q: What does the Supervisor do in your project? It can read and move any object. We use it only for checks (world vs config), screenshots, and later for testing — not for the robot's own decisions.

Q: What is the time step? 16 ms. Every 16 ms of simulated time, physics updates and our controller runs once.

Q: What is left to do? Phase 4 (verify LiDAR direction, test the detector on many frames), Phase 5 (tune the path follower), and for the final evaluation: real pickup, dynamic re-planning, and the optional pen trail.

10. Files changed in this work

File	What changed
tools/generate_world.py	rewritten: fixes 1–6 and 8, writes 2 worlds
worlds/warehouse.wbt, worlds/warehouse_drive_test.wbt	generated worlds (new)
controllers/drive_test/drive_test.py	rewritten: 7 modes, correct measurements, logs in logs/
controllers/intellibot_controller/config.py	parked-obstacle position, camera viewpoints
controllers/intellibot_controller/waypoint_follower.py	renamed from controller.py (fix 7)
controllers/intellibot_controller/intellibot_controller.py	log file now goes to logs/run.csv
tools/analyze_logs.py	clearer tables, 3-panel odometry plot, kinematics file
docs/evidence/*	screenshots, tables, plot, camera frames
logs/*.csv	raw data from the Webots runs
checklist.md, README.md, docs/webots_findings.md, docs/progress_log.md	updated